ML Final Project Report

江楊弼, 張子恩

2024 年 12 月 21 日

1 Data Pre-processing

The files are transformed into all numerical form in convenience for further usage. We then analyzed missing data percentages for each feature and imputed missing data columns as follows.

1. **Imputation for `season` Column:** It is imputed based on the `date` column for training data. As for testing data of stage 2, we just filled `season` with 2024. And the testing data of stage 1 is imputed in step2.
2. **Imputation for `team_season` Column:** We leveraged the `season` and `team_abbr` columns to impute missing values, filling all entries.
3. **Imputation for `is_night_game` Column:** We imputed approximately 80% of games as `True` for both training and testing datasets.
4. **Imputation of Numerical Values:** It is handled by `XGBoost-Regressor`.
5. **Dropping the `date` Column:** After trying various encoding methods with barely no performance improvement, this column was dropped.
6. **Encoding Non-Numerical Columns:** We tested one-hot, label, and beta encoding. Label encoding was chosen as it provided the best results.
7. **Feature Importance Ranking:** Features were ranked by importance based on tree-based models to discard useless features.

2 Machine Learning Approaches

2.1 Logistic Regression

Feature Transformation First we applied PolynomialFeatures to generate polynomial terms. We tested polynomial transform with `degree` = 1,2 and we found that:

- `degree` = 1: The model underfit the data.

- `degree` = 2: Without proper regularization (by setting `C` = 100), the model overfit the data, achieving training accuracy 1.0. Therefore, we tried to optimize the regularization parameter to prevent overfitting.

Hyperparameter Tuning with Cross-Validation Using LogisticRegression and GridSearchCV, we optimized the regularization parameter `C` and selected the best type of penalty according to cross-validation accuracy. The following hyperparameters were selected:

```
C=0.00001, penalty='l2', solver='liblinear'
```

表 1: Optimal hyperparameters for logistic regression.

Observations and Discussion We found that using L2 penalty need more regularization (smaller `C`) than using L1 penalty. The following table compares the value of (training accuracy - stage 1 private score) using L1, L2 respectively. Note that larger value implies more severe overfitting and smaller value implies better generalization.

Penalty \ C	0.00001	0.0001	0.001	0.005
L1	0.00544	0.00544	0.00544	0.07036
L2	0.04962	0.15845	0.29582	0.38233

2.2 (Soft-Margin) SVM

Hyperparameter Tuning with Cross-Validation Using SVC and GridSearchCV, we optimized the parameter `C`, `degree`, `gamma` and selected the best type of kernel according to cross-validation accuracy. The following hyperparameters were selected:

```
kernel='rbf', C=0.2, gamma=0.001
```

表 2: Optimal hyperparameters for SVM.

2.3 Decision Tree

Hyperparameter Tuning with Cross-Validation Using DecisionTreeClassifier and GridSearchCV, we optimized the parameter `criterion`, `max_depth`, `min_samples_leaf` and `min_samples_split` according to cross-validation accuracy. The following hyperparameters were selected:

```
criterion='entropy', max_depth=4, min_samples_leaf=10, min_samples_split=4
```

表 3: Optimal hyperparameters for decision tree.

2.4 Randomforest

Overview We used RandomForestClassifier to predict game outcomes. The model was optimized through feature selection, hyperparameter tunning using Bayesian Optimization (via Optuna). We used two validation strategies for comparison: cross-validation and using 2022-2023 season's examples as the validation set.

Feature Selection

1. We first trained a preliminary Random Forest model with 100 estimators, setting the feature importance threshold as 0.01, which resulted in removing approximately 40 features. However, this significantly reduced testing accuracy, indicating that some critical features were removed.

2. We then adjusted the threshold to 0.001, which removed only one feature: `is_night_game`, and cross-validation confirmed that it generalized well, avoiding the sharp accuracy decline when more aggressive features were selected.

Hyperparameter Tuning Bayesian Optimization can efficiently explored the parameter space to maximize performance, we tun 1000 iterations to find the following optimal hyperparameters :

```
n_estimators=200, max_depth=8, min_samples_split=15,
min_samples_leaf=5, max_features='sqrt', bootstrap=True
```

表 4: Optimal hyperparameters for the RandomForest model (CV).

```
n_estimators=388, max_depth=8, min_samples_split=19,
min_samples_leaf=1, max_features='log2', bootstrap=True
```

表 5: Optimal hyperparameters for the Random Forest model (2022, 2023).

2.5 XGBoost

Overview We used XGBClassifier with 1000 iterations of Bayesian Optimization for tuning, and the model was evaluated using two validation strategies: 5-fold cross-validation and 2022-2023 season's examples as the validation set. Additionally, bootstrap and bagging techniques were explored in Stage 2 to enhance performance.

Bootstrap and Bagging Results and Configuration

The bootstrap and bagging method achieved top public accuracy in Stage 2 among all strategies. However, as this method was not applied in Stage 1, it is excluded from the final comparison table. With number of models (`n_models`) = 50 and bag size (`bag_size`) = 2500, the stage 2 testing accuracy were:

```
Public Accuracy=0.57392, Private Accuracy=0.54738
```

Observations and Discussion

1. In stage 1, the tiny difference between cross-validation and the 2022—2023 validation set suggested that the model generalized well across both strategies. Cross-validation likely benefited from utilizing more data for training across multiple splits.
2. In stage 2, bootstrap and bagging improved testing accuracy to 0.57392, it is likely due to enhanced model diversity and reduced variance by aggregating predictions. Besides, lower accuracy from 2022—2023 validation set (0.55980) might result from overfitting to validation sets.

```
learning_rate=0.0147, max_depth=9, subsample=0.6494,
colsample_bytree=0.6207, gamma=0.0488, min_child_weight=6, n_estimators=100
```

表 6: Optimal hyperparameters for the XGBoost model(2022,2033).

2.6 LightGBM

Overview We used LGBMCLassifier with Bayesian Optimization for tunning and 3-fold cross-validation for evaluation. Besides, additional bootstrap and bagging are used to generate base learner for voting method mentioned after, with **number of models (`n_models`)** = 500 and **bag size (`bag_size`)** = 0.8 * (original data size) samples per model.

Hyperparameter Tuning Run 100 iterations of Bayesian Optimization with early stopping technique to obtain the following optimal hyperparameters :

```
n_estimators=113, learning_rate=0.07606788990725713, min_child_samples=50,
subsample=0.815632809835552, colsample_bytree=0.9648502072095633,
lambda_l1=1.5757406872596613,lambda_l2=9.225643587989719, num_leaves=11,
```

表 7: Optimal hyperparameters for the LightGBM.

Observations and Discussion

1. LightGBM has extremely fast convergence and it is prone to overfitting.
2. The range of each hyperparameter must be set cautiously to make lgb strike a balance between accuracy and generalization.

2.7 Comparison and Discussion

Scalability is evaluated by the inverse of running time. Stability is evaluated by comparing the difference between validation accuracy and testing accuracy. Suitability is evaluated by the average of public score and private score.

	Logistic Reg	SVM	Decision tree	LightGBM
Interpretability	High	Medium	High	Low
Efficiency	<1 min	1.5-2.5 min	<5 sec	1-1.5 min
Scalability	High	High	Very High	High
Train Accuracy	0.63854	0.60149	0.60319	0.71365
Validation Accuracy	0.57227	0.58175	0.59259	0.60947
Private Score in Stage1	0.58892	0.57985	0.55555	0.56462
Public Score in Stage1	0.59199	0.58295	0.55551	0.57004
Private Score in Stage2	0.53758	0.54493	0.55065	0.53431
Public Score in Stage2	0.59219	0.54734	0.54069	0.53571
Stability in Stage1	Very High	Very High	High	High
Stability in Stage2	High	High	High	Medium
Suitability for Stage1	Very High	High	Low	Medium
Suitability for Stage2	Medium	Medium	Medium	Low

	RF (CV)	RF (2022—2023)	XGB (CV)	XGB (2022—2023)
Interpretability	Medium	Medium	Low	Low
Efficiency	120min	40min	500min	300min
Scalability	Low	Medium	Low	Low
Training Accuracy	0.7813	0.7692	0.6557	0.8669
Validation Accuracy	0.7868	0.7679	0.6557	0.8948
Private Score in Stage 1	0.58568	0.58276	0.57985	0.57563
Public Score in Stage 1	0.58747	0.58037	0.58327	0.58263
Private Score in Stage 2	0.54820	0.56209	0.54330	0.54330
Public Score in Stage 2	0.57724	0.56893	0.55481	0.55980
Stability in Stage 1	Medium	Medium	Medium	Medium
Stability in Stage 2	Medium	Medium	Medium	Medium
Suitability for Stage 1	High	High	High	High
Suitability for Stage 2	High	High	Medium	Medium

2.8 The Best One

In Stage 1, the Random Forest(CV) and XGBoost(CV) models are two of our models with the accuracy above 0.58. Combining these two models using soft voting slightly improved the public and private accuracy to 0.58779 and 0.58632. In stage 2, the RandomForest model(2022, 2023) achieved the best result, with public and private testing accuracy 0.56893, 0.56209 respectively.

Possible Reasons for Improved Accuracy with Soft Voting (Stage 1)

1. **Broader Patterns:** Stage 1 spans 2016—2023, requiring models to generalize across historical trends. Cross-validation effectively captures this range of patterns.

2. **Reduced Bias:** Averaging results minimizes extreme errors from individual models.

Possible Reasons for Best Accuracy with RF(2022, 2023) model (Stage 2)

1. **Temporal Relevance:** Using 2022—2023 as the validation set aligns better with 2024 patterns, reflecting recent player performances, team strategies, and rule changes.

2. **Reduced Noise in Validation Data:** 2022—2023 validation avoids mixing older, less relevant patterns (e.g., 2016—2021) into the evaluation process.

3. **Robustness of RF:** The simpler structure of RF is less sensitive to overfitting compared to XGBoost, making it better suited for generalizing to 2024 games.

3 More On Machine Learning Approaches

3.1 Hybrid Model of Decision Tree and SVM

Intuition

To achieve higher performance, we came to a thought of integrating two different powerful approaches. After trying several combinations, we found that using decision tree and SVM is the best.

Model 1: Decision Tree Feature Transformation

First we trained a decision tree on the training data and used the binary representation of leaf nodes as new features. Next, we applied SVM to train on the transformed feature space. The testing accuracy of this model is as follows.

	Stage 1 Private	Stage 1 Public	Stage 2 Private	Stage 2 Public
Score	0.58240	0.57907	0.54738	0.52906

Model 2: SVM in Decision Tree Leaves

First we traiedn a decision tree on the training data, and we limited the least number of examples in each leaf node. Next, we trained a separate SVM classifier on the examples in each leaf node. For each testing example, we routed it using the decision tree to a leaf node and classified it using the corresponding SVM. The testing accuracy of this model is as follows.

	Stage 1 Private	Stage 1 Public	Stage 2 Private	Stage 2 Public
Score	0.56022	0.57842	0.55493	0.55734

3.2 Voting

When additional models such as SVM, LR, CatBoost, and LightGBM (all with testing accuracy above 0.55) were included in the voting technique, the accuracy did not improve much. Moreover, it did not outperform best model in 2.8 (uniformly voting of RandomForest and XGBoost).

Possible Reasons for Lack of Improvement when Blending Models

1. **Lower Accuracy Contribution:** Weaker models diluted the ensemble's overall performance.
2. **Limited Diversity:** The additional models did not provide significantly new information beyond what Random Forest and XGBoost already captured.

Soft voting worked well with complementary models, but adding less competitive models might introduce noise with no meaningful gains.

4 Work Load

	Job description
江楊弼	Data pre-processing except pitcher-filling, LR, RF, XGB, CatBoost, Best model, Voting, Writing the first draft of above mentioned
張子恩	Logistic regression, SVM, Decision tree, Comparison table, Hybrid models (tree + SVM), Voting, Proofreading, Typesetting
葉富銘	Data pre-processing (pitcher-filling), XGB, LGBM, Stacking (XGB + LGBM), Voting, Proofreading, Typesetting
林佑丞	